

Variómetro para vuelo en parapente: Detector de Velocidad Vertical en Tiempo Real

Un **variómetro** para vuelo en parapente es un sistema que provee **feedback auditivo y visual** en tiempo real informando al piloto sobre su velocidad vertical, ya sea de ascenso o descenso. Está basado en la medición de cambios en la **presión atmosférica**, los cuales son inversamente proporcionales a los cambios en la **altitud** del piloto. El propósito de este sistema es facilitar la **detección de corrientes de aire ascendente** para el piloto, las cuales permiten ganar altura para luego planear largas distancias. El sistema contendrá un menú principal donde se podrán ajustar configuraciones o iniciar/frenar el “modo variómetro”. Para esto se utilizará un buzzer pasivo para emitir los sonidos necesarios, una pantalla LCD, un sensor de presión atmosférica y un encoder rotativo con pulsador para navegar el menú.

Objetivos del Proyecto

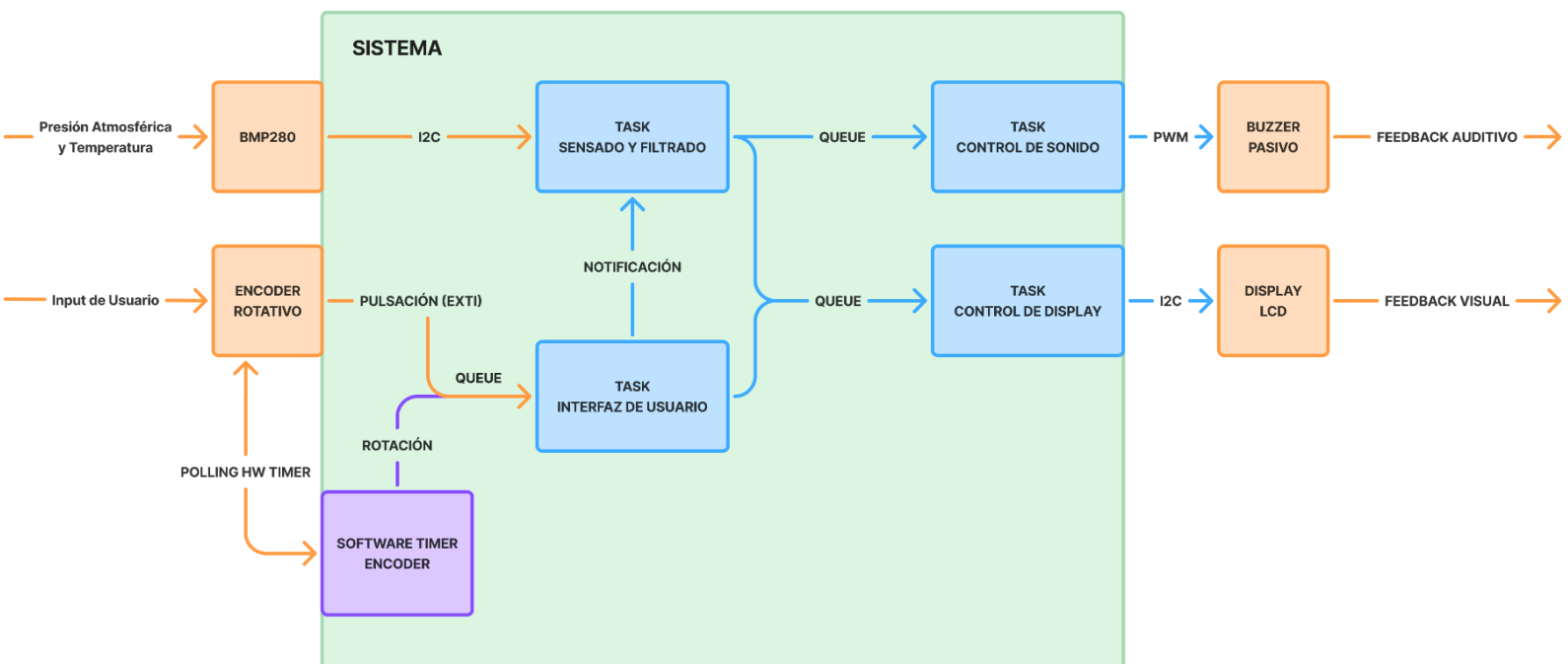
El objetivo principal del proyecto es desarrollar un sistema de variómetro para parapente con feedback visual y auditivo en tiempo real, configurable según las preferencias del piloto.

Objetivos específicos

- ❖ **Implementar un filtro de kalman** para la medición de presión y tasa de ascenso con parámetros ajustables.
- ❖ **Garantizar un tiempo de reacción** y de emisión de sonido acorde a los parámetros seleccionados por el usuario (hasta <200ms en el modo más sensible).
- ❖ **Desarrollar un sistema de feedback** auditivo capaz de variar dinámicamente la frecuencia y cadencia del sonido en función de la tasa de ascenso/descenso.
- ❖ **Construir una interfaz de usuario** interactiva para la visualización de datos de vuelo y un menú principal para la configuración de parámetros del sistema (tonos, umbrales, sensibilidad).

Descripción Funcional

Diagrama de Bloques



Entradas:

- ❖ **Sensor de Presión Atmosférica y Temperatura (BMP280):** Fuente de datos principal para el cálculo de altitud y velocidad vertical. Se lee a través de un bus de comunicación **I2C**.
- ❖ **Rotación del Encoder:** Utilizada por el usuario para la navegación del menú y configuración de parámetros. Se lee mediante un **Timer de hardware** configurado en modo **Encoder**.
- ❖ **Pulsador del Encoder:** Utilizado por el usuario para la selección de opciones y cambio de modos (Ej. salir del modo vuelo). Se detecta mediante una entrada **GPIO configurada con interrupciones (EXTI)**.

Salidas:

- ❖ **Feedback Auditivo (Buzzer Pasivo):** Actuador principal del sistema que emite los tonos de ascenso/descenso en tiempo real. Es manejado mediante una salida de **PWM** generada por un **Timer de hardware** para controlar su frecuencia.
- ❖ **Interfaz de Usuario (Display LCD):** Interfaz para mostrar el menú de configuraciones y los datos de vuelo en tiempo real. Se actualiza mediante comandos enviados a través de un bus **I2C**.

¿Qué hace que sea un sistema de tiempo real?

En un vuelo de parapente, un piloto necesita identificar lugares donde el aire asciende. El parapente está siempre avanzando horizontalmente, típicamente a una velocidad de 30km/h (~9m/s) y tiene una circunferencia de giro (no-agresivo) de 20 a 30m.

Las corrientes de aire que más interesan son las llamadas “térmicas”. Estas pueden imaginarse como “burbujas” o “cilindros” de aire que se desprenden del suelo, y no suelen ser muy anchas comparado al diámetro de giro del parapente.

Si el sistema detecta la entrada a una térmica 1 o 2 segundos tarde, el piloto comenzará a girar para intentar mantenerse en ella pero será muy tarde: parte de su giro se saldrá de la térmica. Una vez afuera, al estar en el aire y venir de un giro, se pierde la referencia de dónde estaba la térmica.

Es decir, el tiempo de respuesta es una parte fundamental de la corrección del sistema, además de la importancia de que los datos reflejen la realidad.

Arquitectura y diseño en FreeRTOS

Tareas Principales

- ❖ **Tarea de Interfaz de Usuario:**
 - Prioridad baja, se requiere tiempo de respuesta “humano”.
 - Responde a eventos de un encoder rotativo, permitiendo la navegación de un menú, cambios de configuraciones e inicio/frenado de vuelos.
- ❖ **Tarea de Sensado y Filtrado:**
 - Prioridad máxima para garantizar frecuencia de muestreo constante.
 - Lee los datos del sensor de presión atmosférica y aplica un filtro de Kalman a una frecuencia constante.
- ❖ **Tarea de Control de Sonido:**
 - Segunda prioridad más alta. Luego de que se produzca un dato por la tarea de sensado, lo primero que debe ocurrir es el feedback auditivo. No es prioridad máxima ya que si la tarea de sensado está leyendo un nuevo dato, conviene esperar antes de emitir un sonido que quizás ya no es relevante.
 - Esta tarea emite los sonidos a distintas frecuencias y cadencias que indican ascenso/descenso al piloto a partir de los datos obtenidos del sensor.
- ❖ **Tarea de Control de Display:**
 - Prioridad mínima, tiempo de respuesta “humano”.
 - Esta tarea recibe eventos de actualización de menú o datos de vuelo y muestra lo requerido en el panel LCD.
- ❖ **Extra: Software Timer para detectar rotaciones del encoder**
 - Modo **AutoReload** con período de **50ms**
 - Asociado a una función de callback que lee el valor del timer asociado al encoder e informa al menú ante eventos de rotación.
 - Se puede **deshabilitar** en modo de vuelo para reducir jitter ya que las rotaciones sólo interesan en el menú principal y **rehabilitar** tras salir del modo de vuelo mediante una pulsación larga.

Sincronización y Comunicación

Una vez iniciado el “modo de vuelo”, las tareas de control de sonido y de control de display dependen exclusivamente de los datos generados por la tarea de sensado y filtrado (se bloquean esperando estos datos en una **queue**), y esta última se encargará de enviarles siempre el dato más reciente mediante el servicio **xQueueOverwrite**.

Se utilizará una **queue de eventos** a través de la cual se bloquea la tarea de interfaz de usuario hasta recibir eventos de rotación, pulsación larga o pulsación corta del encoder para luego procesarlos según corresponda.

La tarea de control de display recibirá distintos comandos mediante una **queue**. Cada comando puede tener datos asociados o no (prender, apagar, mostrar menú, mostrar datos de vuelo, etc) y estos comandos pueden ser enviados desde la tarea de interfaz de usuario o la tarea de sensado, dependiendo el estado del sistema.

Se utilizarán **Direct to Task Notifications** (o un semáforo binario) para el inicio/frenado de la tarea de sensado.

Interrupciones

La principal fuente de interrupciones será el pulsador del encoder, en donde se generarán interrupciones de GPIO ante Falling y Rising para así poder calcular su duración y distinguir entre eventos de pulsación larga y “clicks”. Sus callbacks serán sencillas y utilizarán el servicio **xQueueSendFromISR** para registrar los eventos detectados.

Arquitectura de Hardware

El microcontrolador utilizado será el **STM32 Nucleo H533RE**.

Los periféricos configurados serán:

- ❖ **Timer en modo encoder** (lee dos canales para detectar **rotaciones** del encoder en cualquier sentido).
- ❖ Un pin de **GPIO en modo EXTI** de Rising y Falling para las **pulsaciones** del encoder.
- ❖ **Timer en modo PWM** para la generación de distintas frecuencias de **sonido** en el **buzzer**.
- ❖ **I²C** para la lectura del **sensor** y la escritura al **display**. En principio serán **dos buses** independientes para aislar la lógica de 3.3v (sensor) de la de 5v (LCD), pero en caso de utilizar uno solo, será protegido por **mutex**.

Métodos de validación

Un variómetro es lo suficientemente sensible como para generar sonidos ante pequeños cambios de altitud sosteniéndolo en la mano. Este será el principal método de validación ya que el cumplimiento de los requisitos de tiempo real del sistema es verificable por un humano (una persona puede notar la diferencia entre 100ms y 1500ms de respuesta).

También tengo a mi disposición variómetros para parapente “profesionales” por lo que se podría realizar una comparación con estos sistemas ya validados por miles de pilotos.